

Segmentação e Reconhecimento de Rótulos em Embalagens de Frango

Documento técnico: arquitetura, requisitos, dados, testes e resultados

Bruno Silveira (@cenourin)

Contents

1 Visão geral

Este documento descreve a arquitetura e os resultados do projeto de Processamento Digital de Imagens (PDI) composto por duas etapas sobre o mesmo conjunto de fotos de embalagens de produtos avícolas, capturadas em uma esteira industrial com câmera fixa:

- **T1 — Segmentação:** localizar e recortar a região do rótulo com o nome do produto em cada imagem de embalagem.
- **T2 — Reconhecimento:** identificar a qual classe de produto pertence um segmento gerado no T1, comparando-o com um template de referência por classe.

Ambas as etapas usam exclusivamente técnicas clássicas de PDI (espaços de cor, limiarização, morfologia matemática, filtragem espacial, transformações geométricas, descritores locais de características) — sem uso de aprendizado de máquina ou classificadores treinados em nenhuma das duas etapas.

2 Requisitos

2.1 Requisitos funcionais

1. RF01 — Dado um caminho de imagem de embalagem, o pipeline de T1 deve retornar um recorte quadrado contendo a região do nome do produto, ou nenhum recorte quando não há candidato confiável.
2. RF02 — O pipeline de T1 deve tratar dois layouts físicos distintos de embalagem (bandeja e selado), cada um com um pipeline próprio.
3. RF03 — Dado um segmento de T1 (ou um recorte manualmente corrigido) e um conjunto de templates gold (um por classe), o pipeline de T2 deve retornar a classe de maior correspondência ou *unknown* quando nenhum template atinge o limiar mínimo de confiança.
4. RF04 — Ambos os pipelines devem processar o dataset inteiro em lote e produzir um resumo por classe (sucesso/falha em T1; acerto/não identificado/falso positivo em T2).

2.2 Requisitos não funcionais

1. RNF01 — **Zero alteração de código em tempo de avaliação:** os parâmetros dos pipelines são centralizados (dicionário CFG em T1; `RecognitionConfig` em T2) e devem generalizar para um conjunto de imagens novo, não apenas para as imagens vistas durante o desenvolvimento.
2. RNF02 — **Nenhuma biblioteca de segmentação ou classificação baseada em IA/ML:** apenas operações clássicas de PDI (OpenCV: limiarização, morfologia, contornos, SIFT/ORB + FLANN para correspondência de descritores).

3. RNF03 — **Falso positivo é mais custoso que falso negativo**: em ambas as etapas, a calibração de parâmetros prioriza reduzir a taxa de falso positivo mesmo à custa de recall (mais recortes/classificações rejeitados como inválidos ou *unknown*).

3 Dados

O dataset (`Train_and_Validation/`) contém 18 classes de produto, cada uma em uma pasta nomeada `<código>_<Nome_Do_Produto>`, com aproximadamente 50 fotos por classe tiradas por um rig de câmera fixo sobre a esteira de produção. As classes se dividem em dois tipos de embalagem física, que exigem pipelines de segmentação diferentes:

- **Bandeja**: rótulo como região destacada e não fixa dentro do quadro (7 classes usadas na etapa de reconhecimento do T2).
- **Selado**: rótulo em uma faixa horizontal fixa da embalagem, com uma região de FPS/timestamp sobreposta que precisa ser mascarada (11 classes).

O dataset completo não é versionado neste repositório (ver `README.md`) por ser grande demais e por conter imagens de produto de uma marca comercial (Super Frango) cujos direitos não pertencem ao autor deste trabalho.

4 Arquitetura — T1 (Segmentação)

Pipeline linear por imagem, cada etapa uma função pura operando sobre a saída da etapa anterior, com todos os parâmetros centralizados em um dicionário de configuração em vez de espalhados como literais no código.

4.1 Pipeline bandeja (`segmentador_colab.ipynb`)

1. `extrair_brilho` — conversão BGR→LAB e CLAHE no canal L, para compensar iluminação industrial desigual.
2. `suavizar` — filtro de mediana (remoção de ruído).
3. `binarizar` — limiarização de Otsu com deslocamento manual (`thresh_offset`): Otsu sozinho tende a escolher um limiar entre o texto escuro do rótulo e o fundo mais claro, então é ajustado empiricamente.
4. `aplicar_roi` — zera uma margem fixa de borda (esteira/paredes laterais ficam fora de `roi_left/right/top/bot`).
5. `morfologia` — fechamento (junta o texto do rótulo em um blob) seguido de abertura (remove blobs de ruído pequenos).
6. `classificar_regioes` — extração de contornos com filtros geométricos (área, proporção largura/altura, altura mínima, solidez), anotando o motivo de rejeição de cada região descartada.
7. `nms` — supressão de sobreposição por IoU entre caixas candidatas; apenas a maior caixa aceita é mantida.
8. `square_crop` — recorte quadrado centralizado na caixa vencedora (`lado = max(largura, altura)`, sem redimensionamento).

4.2 Pipeline selado (`segmentador_selado.ipynb`)

Layout físico diferente do bandeja (rótulo em faixa horizontal fixa em vez de região solta): alongamento de contraste por percentil → recorte de ROI com máscara fixa da região de FPS/timestamp → morfologia top-hat → limiarização adaptativa/fixa → componentes conectados filtrados por área → *clustering* espacial de componentes para reagrupar texto multi-letra em uma única região, preferindo o agrupamento de proporção larga (horizontal), que é a orientação do texto nesse tipo de embalagem.

5 Arquitetura — T2 (Reconhecimento)

1. **Templates:** um recorte manual por classe (região do rótulo com maior potencial discriminativo), armazenado em `t2/templates-gold/`.
2. **Extração de características:** SIFT (`cv2.SIFT_create`), com ORB disponível como alternativa via configuração.
3. **Casamento:** `cv2.FlannBasedMatcher`, com teste de razão de Lowe aplicado nos dois sentidos (cross-check).
4. **Verificação geométrica:** RANSAC com transformação afim sobre os matches válidos — descarta correspondências espúrias em elementos repetidos entre rótulos (marca, palavra "CONGELADO", tabela de peso), a principal fonte de falso positivo nesse tipo de matching.
5. **Score:** fração de matches válidos (inliers) sobre o número de pontos de interesse do template. O template de maior score vence, desde que o score atinja o limiar `min_match_frac`; caso contrário, o segmento é reportado como *unknown*.

Detalhamento de parâmetros e a varredura de calibração de `min_match_frac` estão em `docs/relatorio-t2.tex`.

6 Metodologia de teste

6.1 T1

Execução em lote sobre o dataset completo, com contagem por classe de sucesso (recorte gerado) e falha, e motivo de rejeição registrado por região descartada (área insuficiente, proporção fora da faixa, altura mínima não atingida, baixa solidez). Falso positivo (recorte sem o texto do produto, ou com o texto errado) é tratado como erro mais grave que detecção perdida, conforme o enunciado da disciplina.

6.2 T2

Avaliação sobre 350 imagens (7 classes bandeja $\times$ 50 imagens), usando os segmentos gerados no T1 e corrigidos manualmente quando o T1 não gerou recorte. Métricas: acurácia (classificação correta), taxa de não identificado (*unknown*, quando nenhum template atinge o limiar) e taxa de falso positivo (classificado com produto errado). O parâmetro `min_match_frac` foi calibrado por varredura, buscando o menor valor que mantém a taxa de falso positivo baixa sem descartar recall demais — ver tabela de calibração em `docs/relatorio-t2.tex`.

7 Análise de resultados

7.1 T1

Amostras de resultado (uma imagem por classe, grids com os recortes gerados) estão em `docs/resultados-exem` (7 classes) e `docs/resultados-exemplo/selado/` (11 classes) — as 18 classes do dataset. A saída completa do pipeline (todas as imagens, todas as classes) não é versionada; pode ser regenerada rodando os notebooks sobre o dataset local.

7.2 T2

Com `min_match_frac = 0,08`: acurácia global de 83,4% (292/350), 48 segmentos não identificados e 10 falsos positivos (2,9%) sobre as 350 imagens avaliadas. O desempenho varia por classe — produtos com texto curto e exclusivo (Coração, Filé de Peito) atingem 100% de acerto, enquanto classes com rótulos de menor contraste ou mais elementos repetidos entre embalagens (Filé de Coxas e Sobrecoxas com Pele, Coxinhas das Asas) têm mais segmentos

descartados como *unknown*. Detalhamento completo por classe e a matriz de confusão estão em docs/relatorio-t2.tex e docs/matriz_confusao_t2.png.

8 Aplicação em um caso real

O pipeline desenvolvido é um protótipo direto de um sistema de **controle de qualidade em linha de produção** para indústria alimentícia: uma câmera fixa sobre a esteira de embalagem captura cada unidade, o T1 localiza e recorta o rótulo, e o T2 confirma se o rótulo corresponde ao produto esperado para aquela linha/turno. Um *unknown* ou uma classificação divergente da SKU configurada pode disparar um alerta ou desviar a unidade da linha antes do despacho, evitando o erro de expedir produto com rótulo trocado.

A prioridade de projeto adotada em ambas as etapas — minimizar falso positivo mesmo à custa de recall — reflete diretamente o custo assimétrico desse cenário real: deixar passar uma embalagem sem verificação automática (falso negativo, cai para inspeção manual) é muito mais barato que confirmar automaticamente um rótulo errado como correto (falso positivo, produto sai da fábrica mal rotulado).

Limitações para uso em produção: os parâmetros de segmentação foram calibrados para as condições de iluminação e o rig de câmera específicos deste dataset; uma implantação real exigiria recalibração para cada linha/câmera, e o tempo de processamento por imagem (SIFT + RANSAC) precisaria ser validado contra a cadência real da esteira.

9 Direitos autorais

As imagens do dataset utilizado neste trabalho retratam embalagens e produtos da marca **Super Frango**, cujos direitos pertencem aos seus respectivos titulares. As imagens foram disponibilizadas exclusivamente para fins acadêmicos nesta disciplina. Este documento e o repositório associado versionam apenas o código de processamento de imagem desenvolvido pelo autor; nenhuma reivindicação de direito é feita sobre a marca, o produto ou as imagens originais do dataset, e o autor não se responsabiliza pelo uso de tais imagens fora do escopo acadêmico deste trabalho.